

Day_14_GAN

September 21, 2026

Week_03_Day_13: Quantum GAN: RAJIV KUMAR YADAV

```
[1]: import numpy as np

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from scipy.optimize import minimize

# =====
# 1. TARGET DISTRIBUTION
# =====

target_distribution = np.array([0.8, 0.2])

# =====
# 2. QUANTUM GENERATOR
# =====

def generator(theta):

    theta = float(theta[0])

    qc = QuantumCircuit(1)

    # Parameterized quantum circuit
    qc.ry(theta, 0)

    return qc

# =====
# 3. GET GENERATED PROBABILITIES
# =====

def generated_distribution(theta):
```

```

qc = generator(theta)

simulator = AerSimulator(
    method="statevector"
)

qc.save_statevector()

result = simulator.run(qc).result()

state = np.asarray(
    result.get_statevector()
)

probabilities = np.abs(state) ** 2

return probabilities

# =====
# 4. GENERATOR LOSS
# =====

def loss(theta):

    generated = generated_distribution(theta)

    # Mean squared error
    error = np.mean(
        (generated - target_distribution) ** 2
    )

    return error

# =====
# 5. INITIAL PARAMETER
# =====

initial_theta = [0.5]

# =====
# 6. TRAIN THE GENERATOR
# =====

result = minimize(

```

```

    loss,
    initial_theta,
    method="COBYLA",
    options={"maxiter": 100}
)

# =====
# 7. RESULTS
# =====

optimal_theta = result.x[0]

generated = generated_distribution(
    result.x
)

print("\n=====")
print("          QGAN RESULT")
print("=====")

print("Optimal theta:", optimal_theta)

print("\nTarget distribution:")
print(target_distribution)

print("\nGenerated distribution:")
print(generated)

print("\nLoss:")
print(result.fun)

print("=====")

```

```

=====
          QGAN RESULT
=====
Optimal theta: 0.92725

Target distribution:
[0.8 0.2]

Generated distribution:
[0.80001809 0.19998191]

Loss:

```

3.2713573234481087e-10

=====

[]: